

# Solving partial ordering issues introduced by P0522R0

Document #: P3310R0  
Date: 2024-05-21  
Project: Programming Language C++  
Audience: EWG, CWG  
Reply-to: Matheus Izvekov  
<[mizvekov@gmail.com](mailto:mizvekov@gmail.com)>

## Contents

- [1 Introduction](#)
- [2 Problem #1 - Default Arguments](#)
  - [2.1 Solution to #1](#)
  - [2.2 Wording for #1](#)
- [3 Problem #2 - With vs Without Packs](#)
  - [3.1 Solution to #2](#)
  - [3.2 Wording to #2](#)
- [4 References](#)

## § 1 Introduction

This paper aims to address some lingering issues introduced with the adoption of P0522R0 into C++17, later made a defect report, which relaxed the rules on how templates are matched to template template parameters, but didn't concern itself with how partial ordering would be affected.

As a result, it invalidated several perfectly legitimate prior uses, creating compatibility issues with old code, forcing implementors to amend the rules, and overall slowing adoption.

We will point out two separate issues and their proposed solutions, which can be picked and voted independently, with the intention that they be adopted as resolutions to [CWG2398], which is the core issue tracking this problem.

The intent is to keep old code working, not to change best practices. With that said, these proposed rules should keep things more consistent and explainable, as candidates having

template template parameters with more non-pack parameters will be more specialized than those with fewer, and those taking pack parameters will be less specialized than those that don't.

If any of the solutions is adopted, it is suggested to bump the version for the feature testing macro.

## § 2 Problem #1 - Default Arguments

Consider the following example:

```
template<class T1> struct A;  
  
template<template<class T2> class TT1, class T3>  
struct A<TT1<T3>>; // #1  
  
template<template<class T4, class T5> class TT2, class T6, class T7>  
struct A<TT2<T6, T7>> {}; // #2  
  
template<class T8, class T9 = float> struct B;  
template struct A<B<int>>;
```

Prior to [P0522R0], with the more strict rules, only the partial specialization candidate #2 would be considered, since B has two template parameters, T8 and T9, and the template template parameter in #1 has only T2 as a template parameter, so #1 doesn't match.

After P0522R0, #1 starts being accepted as a candidate, since B can be used just fine in place of TT1 in the specialization A<TT1<T3>>. The default argument `float` for T9 allows B to be specialized with just a single argument.

However, the rules for partial ordering operate considering only the candidates themselves, without access to B, which has the default template argument. When looking at only those candidates, it's not obvious the specialization of TT1 would work, if that were to be replaced with a template with two parameters.

The only clue is the fact that these two candidates are being considered together: this must mean that default arguments are somehow involved.

But nonetheless, as things stand, these candidates cannot be ordered, resulting in ambiguity.

Maintaining the pre-P0522 semantics of picking #2 would be ideal: Besides that it would be a compatible change, it is logical that this is the most specialized candidate.

### § 2.1 Solution to #1

When performing template argument deduction such that P and A are specializations of either class templates or template template parameters:

```
P = TT1<X1, ..., Xn>
A = TT2<Y1, ..., Yn>
```

Under the current rules, TT1 will be deduced as TT2.

This paper proposes that in this case, TT1 will be deduced as a new template, synthesized from and almost identical to TT2, except that the specialization arguments (Y1, ..., Yn) will be used as default arguments in this new invented TT2, replacing any existing ones, effectively as if it had been written as:

```
template<class T1 = Y1, ..., class Tn = Yn> class TT2
```

Conversely, a template template parameter can also be deduced as a class template:

```
template <class T1, class T2 = float> struct A;

template <class T3> struct B;
template <template <class T4> class TT1, class T5> struct B<TT1<T5>>; //
#1
template <class T6, class T7>
#2                                struct B<A<T6, T7>>; //

template struct B<A<int>>;
```

When partially ordering #1 versus #2, TT1 will be deduced as a synthesized template re-declaration of A with <T6, T7> as default arguments.

When these proposed rules are applied to class template deduction, this example, which is unrelated to partial ordering, also becomes valid:

```
template<class T, class U> struct A {};
A<int, float> v;
template<template<class> class TT> void f(TT<int>);
void g() { f(v); }
```

Here, ‘TT’ picks a synthesized re-declaration of ‘A’ which has ‘float’ as the default argument for the second parameter.

When a parameter is deduced against multiple arguments, consistency must be enforced.

Given this example:

```
template<class T> struct N { using type = T; };

template<class T1, class T2, class T3> struct A;

template<template<class, class> class TT1,
        class T1, class T2, class T3, class T4>
struct A<TT1<T1, T2>, TT1<T3, T4>,
        typename N<TT1<T1, T2>>::type> {};

template<template<class> class UU1,
        template<class> class UU2,
        class U1, class U2>
```

```
struct A<UU1<U1>, UU2<U2>, typename N<UU1<U1>>::type>;

template<class T8, class T9 = float> struct B;
template struct A<B<int>, B<int>, B<int>>>;
```

This was accepted prior to P0522R0, but picking `<T1, T2>` for the default arguments of `TT1` will result in this example being accepted, but result in rejection of the symmetric example where `N<TT1<T1, T2>>` is replaced with `N<TT1<T1, T4>>`, which is also accepted pre-P0522.

There is no known mechanically simple and sound way to solve this. It would not be simple, but sound, if the solution were to include a mechanism to deduce that `T2` and `T4` must be the same type. Unless there is sufficient motivation to try to salvage this situation, it's proposed that both forms be rejected, for consistency.

To achieve this consistency, multiple deductions must be combined by stripping out differing default arguments.

## § 2.2 Wording for #1

Modify §13.10.3.6 13.10.3.6 [\[temp.deduct.type\]](#)/8:

- 8 A template type argument `T`, a template template argument `TT`, or a template non-type argument `i` can be deduced if `P` and `A` have one of the following forms:

```
cvopt T
T*
T&
T&&
Topt [iopt]
Topt (Topt) noexcept(iopt)
Topt Topt::*
TTopt<T>
TTopt<i>
TTopt<TTopt>
TTopt<>
```

- (8.1) - `Topt` represents a type or parameter-type-list that either satisfies these rules recursively, is a non-deduced context in `P` or `A`, or is the same non-dependent type in `P` and `A`,
- (8.2) - `TTopt` represents either a class template or a template template parameter,

[Example:

```
template<class T, class U> struct A {};
template<template<class> class TT> void f(TT<int>);
```

```
void g() {
    A<int, float> v;
    // OK, TT = A, 'float' is used as default argument the second parameter
    f(v);
}
```

— end example]

- (8.3) - When matching  $TT_{\text{opt}}$  specializations,  $TT_{\text{opt}}$  will be deduced as if its template argument list in A were used as the default arguments for its template parameters, up to the last parameter which has a corresponding argument. When multiple such arguments are deduced, only default arguments which are common to all deductions are kept.

Append examples to §13.4.4 13.4.4 [\[temp.arg.template\]/4](#):

[Example:

```
template <class, class = float> struct A;

template <class> struct B;
template <template <class> class TT, class T> struct B<TT<T>>; // #1
template <class T, class U> struct B<A<T, U>>; // #2
template struct B<A<int>>; // selects #2

template <class> struct C;
template <template <class> class TT, class T> struct C<TT<T>>; // #3
template <class T, class U> struct C<A<T, U>>; // #4
template struct C<A<int>>; // selects #4
```

— end example]

[Example:

```
template<class T> struct N { using type = T; };

template<class, class = float> struct A;

template<class, class, class> struct B;

template<template<class, class> class TT,
        class T1, class T2, class T3, class T4>
struct B<TT<T1, T2>, TT<T3, T4>,
        typename N<TT<T1, T2>>::type>;

template<template<class> class UU1,
        template<class> class UU2,
        class U1, class U2>
struct B<UU1<U1>, UU2<U2>, typename N<UU1<U1>>::type>;

template struct B<A<int>, A<int>, A<int>>; // ambiguous

template<class, class, class> struct C;

template<template<class, class> class TT,
        class T1, class T2, class T3, class T4>
```

```

struct C<TT<T1, T2>, TT<T3, T4>,
    typename N<TT<T1, T4>>::type>;

template<template<class> class UU1,
    template<class> class UU2,
    class U1, class U2>
struct C<UU1<U1>, UU2<U2>, typename N<UU1<U1>>::type>;

template struct C<A<int>, A<int>, A<int>>; // ambiguous

```

— end example]

Modify example in §13.7.8 13.7.8 [temp.alias]/2:

[Example 1:

```

-13,7 +13,8
template<template<class> class TT>
    void f(TT<int>);

-f(v); // error: Vec not deduced
+// OK, TT = vector, Alloc<int> used as default argument for second
+parameter
+f(v);

template<template<class,class> class TT>
    void g(TT<int, Alloc<int>>);

```

— end example]

## § 3 Problem #2 - With vs Without Packs

Consider the following example:

```

template<template<class ...T1s> class TT1> struct A {};
template<class T2> struct B;
template struct A<B>; // #1

template<template<class T3> class TT2> struct C {};
template<class ...T4s> struct D;
template struct C<D>; // #2

```

Before [P0522R0], #1 was valid, and #2 was invalid.

After P0522R0, #1 stayed valid, and #2 became valid.

Now consider this partial ordering example:

```

template<class T1> struct A;

template<template<class ...T2s> class TT1, class T3>
struct A<TT1<T3>>; // #1

template<template<class T4> class TT2, class T5>

```

```
struct A<TT2<T5>> {}; // #2

template<class T6> struct B;
template struct A<B<int>>>;
```

Before P0522R0, #2 was picked.

However, since the same rules which determine a template can bind to a template template parameter, are also used to determine one template template parameter is at least as specialized as another, and since P0522R0 made no special provisions in these rules for the latter, this example now becomes ambiguous.

But this is undesirable, because it is a breaking change, and also because logically #2 is more specialized.

The new paragraph inserted with P0522R0, which defines the ‘at least as specialized as’ rules in terms of function template rewrite, leads to this confusion:

The template argument list deduction rules accept packs in parameters and no packs in arguments, while the reverse is rejected, but in order to accept both #1 and #2 from the first example, both directions need to be accepted. An exception had to be carved out in 13.4.4 [temp.arg.template]/3 for this.

We propose a solution which will restore the semantics of the second example, by specifying that during partial ordering, packs must match non-packs in only one direction.

Now consider this last example:

```
template <template <class...      > class TT1> struct A      { static
    constexpr int V = 0; };
template <template <class        > class TT2> struct A<TT2> { static
    constexpr int V = 1; };
template <template <class, class> class TT3> struct A<TT3> { static
    constexpr int V = 2; };

template <class ...      > struct B;
template <class          > struct C;
template <class, class   > struct D;
template <class, class, class> struct E;

static_assert(A<B>::V == 0);
static_assert(A<C>::V == 1);
static_assert(A<D>::V == 2);
static_assert(A<E>::V == 0);
```

Before P0522R0, this is accepted.

After P0522R0, this became wholly invalid: The partial specializations are not more specialized than the primary template. Also, the A<B> specialization becomes ambiguous.

With the proposed solution, the primary template becomes less specialized again.

But the other issue will remain: The  $A < B$  specialization will stay ambiguous. A solution to this last problem is left for future work.

## § 3.1 Solution to #2

We propose changing the deduction rules such that, only during partial ordering, packs matching to non-packs isn't accepted both ways, that it should only be accepted in the argument to parameter direction, which is the opposite direction it's normally accepted in the deduction of template argument lists.

We also propose scratching the classic pack exception clause in `[temp.arg.template]/3{sref}`, in favor of explicitly specifying that outside of partial ordering, packs matching to non-packs must be accepted both ways, parameter to argument as well as argument to parameter.

## § 3.2 Wording to #2

Modify §13.10.3.6 13.10.3.6 `[temp.deduct.type]/9`:

- 9 If  $P$  has a form that contains  $\langle T \rangle$ ~~or~~,  $\langle i \rangle$ , or  $\langle TT_{opt} \rangle$ , then each argument  $P_i$  of the respective template argument list of  $P$  is compared with the corresponding argument  $A_i$  of the corresponding template argument list of  $A$ . If the template argument list of  $P$  contains a pack expansion that is not the last template argument, the entire template argument list is a non-deduced context. ~~If  $P_i$  is a pack expansion, then the pattern of  $P_i$  is compared with each remaining argument in the template argument list of  $A$ . Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by  $P_i$ . During partial ordering, if  $A_i$  was originally a pack expansion:~~

(9.1) ~~- if  $P$  does not contain a template argument corresponding to  $A_i$  then  $A_i$  is ignored;~~

(9.2) ~~- otherwise, if  $P_i$  is not a pack expansion, template argument deduction fails.~~

(9.1) Except when deducing the argument list of  $X$  as specified in 13.4.4 `[temp.arg.template]/4`, while checking deduced template arguments, if  $P_i$  is a pack expansion, then the pattern of  $P_i$  is compared with each remaining argument in the template argument list of  $A$ . Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by  $P_i$ . During partial ordering, if  $A_i$  was originally a pack expansion:

(9.1.1) - if  $P$  does not contain a template argument corresponding to  $A_i$  then  $A_i$  is ignored;

(9.1.2) - otherwise, if  $P_i$  is not a pack expansion, template argument deduction fails.

(9.2) When deducing the argument list of  $X$  as specified in 13.4.4 `[temp.arg.template]/4`, if  $A_i$  is a pack expansion, then the pattern of  $A_i$  is compared with each remaining argument in the



template argument list of P. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by  $A_i$ . If  $P_i$  was originally a pack expansion:

- (9.2.1) - if A does not contain a template argument corresponding to  $P_i$  then  $P_i$  is ignored;
- (9.2.2) - otherwise, if  $A_i$  is not a pack expansion, template argument deduction fails.

[Note: (9.2) is the reverse of (9.1) - end note]

Modify §13.4.4 13.4.4 [temp.arg.template]/3:

- 3 A template-argument matches a template template-parameter P when P is at least as specialized as the template-argument A. In this comparison, if P is unconstrained, the constraints on A are not considered. ~~If P contains a template parameter pack, then A also matches P if each of A's template parameters matches the corresponding template parameter in the template-head of P.~~ Two template parameters match if they are of the same kind (type, non-type, template), for non-type template-parameters, their types are equivalent (13.7.7.2 [temp.over.link]), and for template template-parameters, each of their corresponding template-parameters matches, recursively. When P's template-head contains a template parameter pack (13.7.4 [temp.variadic]), the template parameter pack will match zero or more template parameters or template parameter packs in the template-head of A with the same type and form as the template parameter pack in P (ignoring whether those template parameters are template parameter packs).

Append note and example to §13.4.4 13.4.4 [temp.arg.template]/4:

[Note: 13.10.3.6 [temp.deduct.type]/9 has a special case for this deduction - end note]

[Example:

```
template<class T1> struct A;
template<template<class ...> class TT, class T> struct A<TT<T>>; // #1
template<template<class    > class TT, class T> struct A<TT<T>>; // #2

template<class> struct B;
template struct A<B<int>>; // selects #2

template <template <class...> class TT> struct C;
// This partial specialization is more specialized than the primary
//   template.
template <template <class> class TT> struct C<TT>;

template <template <class> class TT> struct D;
// This partial specialization is NOT more specialized than the primary
//   template.
template <template <class...> class TT> struct D<TT>;
```

— end example]

## § 4 References

[CWG2398] Jason Merrill. 2016-12-03. Template template parameter matching and deduction.

<https://wg21.link/cwg2398>

[P0522R0] James Touton, Hubert Tong. 2016-11-11. DR: Matching of template template-arguments excludes compatible templates.

<https://wg21.link/p0522r0>